

Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Neural Networks
MNIST

Oct 15, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt / Fraunhofer IVI

Autonomous, Cooperating Systems

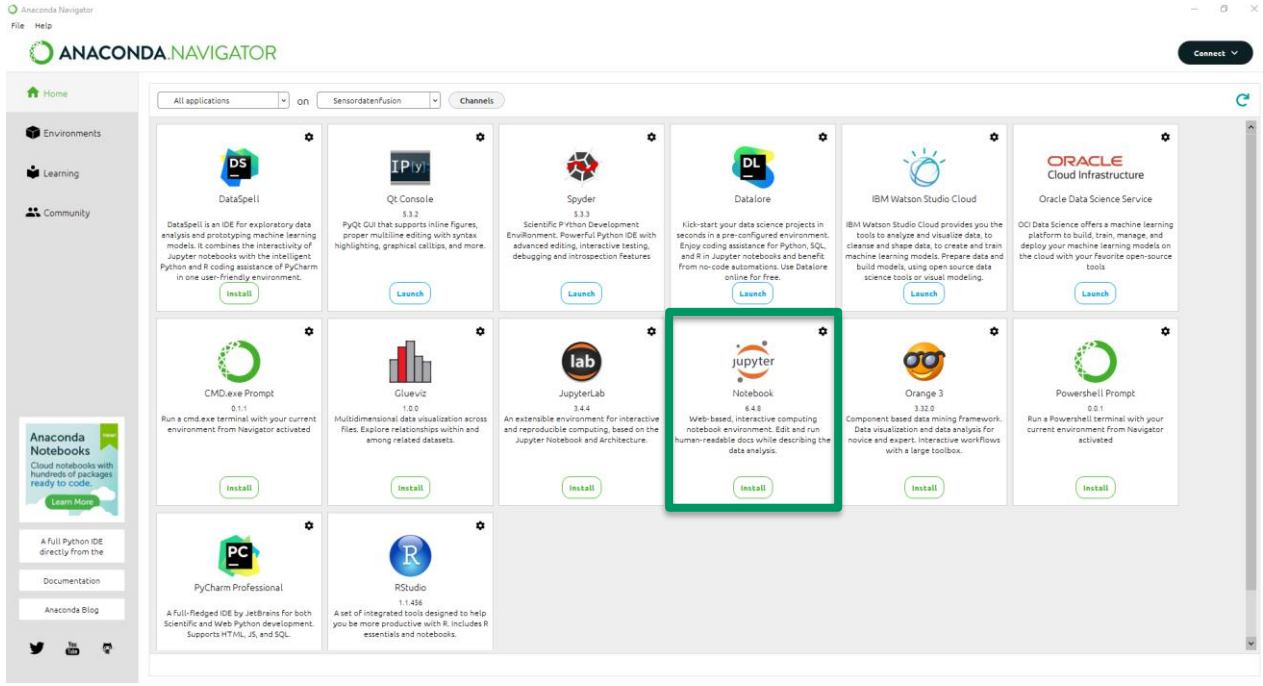
Email: bjoern.giesler@thi.de

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **News on Moodle:**
  - Slides for the lecture (upcoming lectures marked as WIP and are subject to change!)
  - Formulary, Exercises (Solutions provided after discussion in the lecture)
  - Exercises are optional, one task / sheet per week, but are highly recommended.
    They can be handed in for correction.
  - I will *try* to publish exercises before Wednesday‘s lecture so that initial questions can be discussed on Thursday.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

- **You will find two exercises accompanying last week‘s lectures on Moodle.**
- ***Exercise Sheet 1***
  - A couple of very gentle initial exercises (vector/matrix multiplication, Euler angles, derivatives / gradient vectors).
  - Doing this sheet is **highly recommended**, as is handing it in for correction. Depending on number of submissions, I will try to give results on Monday.
- ***ProgExercise 2***
  - A Python program which realizes a Viola Jones face detector.
  - Your task is to use the supplied camera calibration matrix K, and the size of the detected face, to estimate the 3D position of the face in space.
  - This will develop into a longer series of programming exercises estimating positions and velocities of objects in space with Kalman Filters.
  - Doing the exercise therefore is **highly recommended** as well.
- **Solutions for both of the exercises will be posted on Wed Oct 22.**

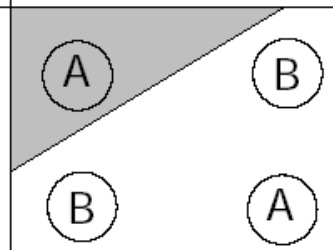
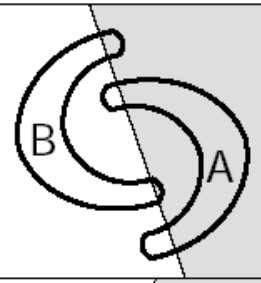
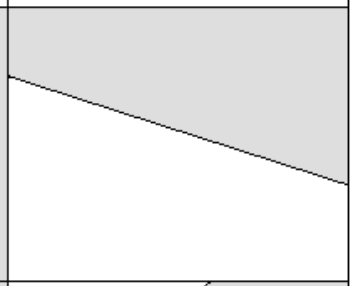
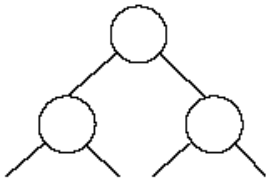
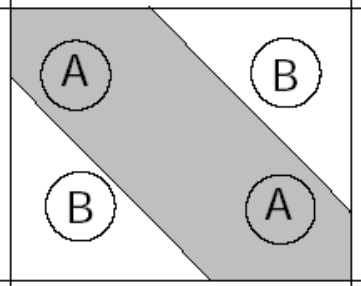
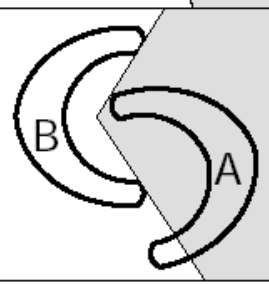
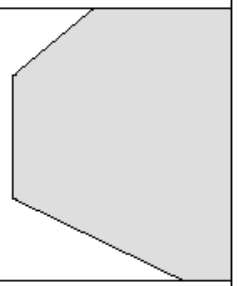
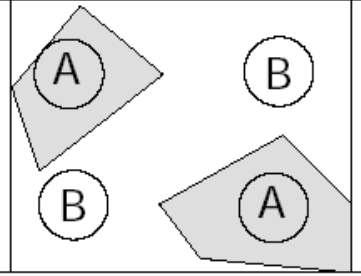
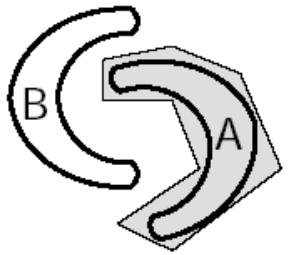
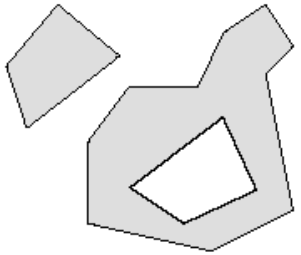
Software: Python/Anaconda

- For the ML examples we will use today, will use the Jupyter Notebook environment.
- It is recommended you install the Anaconda Python environment and distribution, that installs Jupyter as one of its environments.
- You can find it at https://www.anaconda.com/.

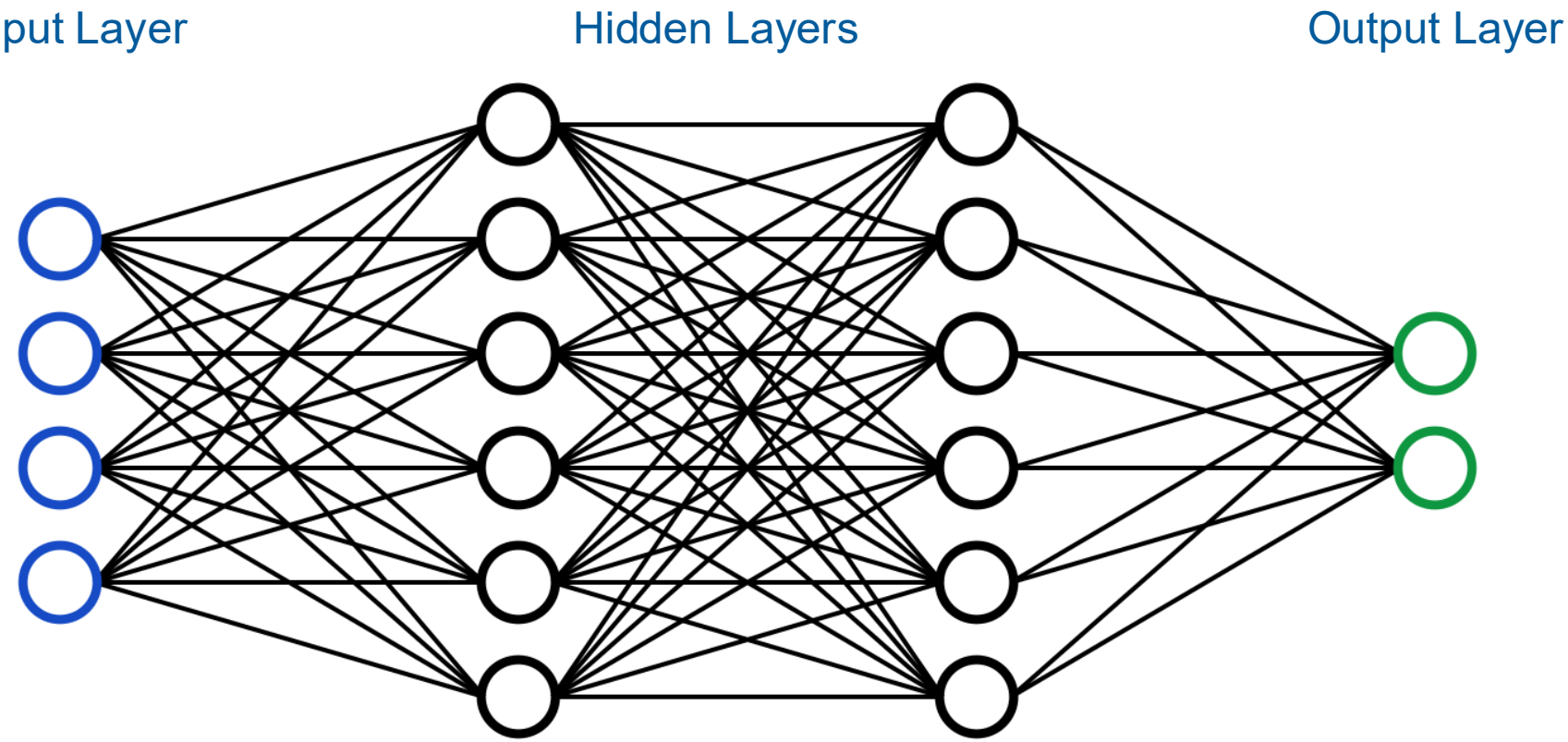


Machine Learning: Building Networks From Neurons

- **A single neuron models a single line (or hyperplane) separator. It can for example not model XOR.**
- **Like in a human brain, we construct multiple layers of neurons for more general separators.**

Network Structure	Separation Region	XOR separation	Nested Region Separation	General Region Separation
One layer 	Single plane			
Two layers 	Convex regions			
Three layers 	Arbitrarily complex regions			

For more difficult problems, we build up multi-layer networks of many neurons.

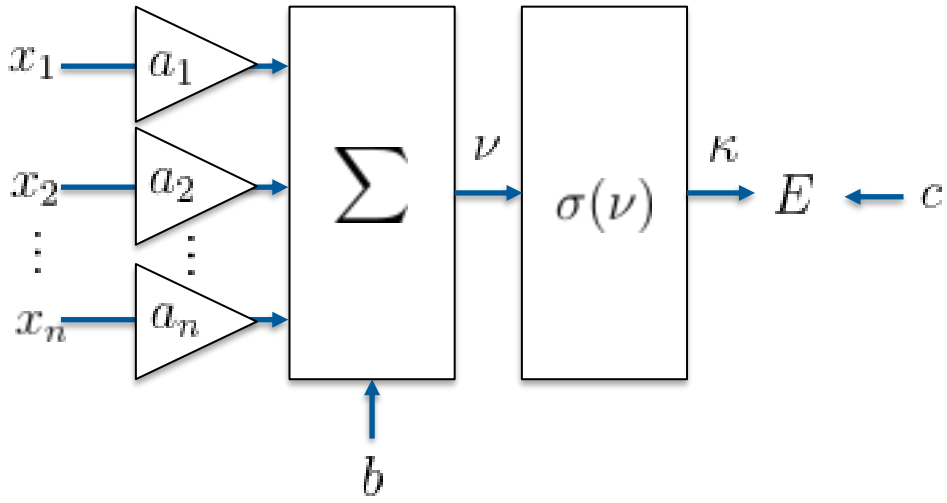


Recap Single Neuron

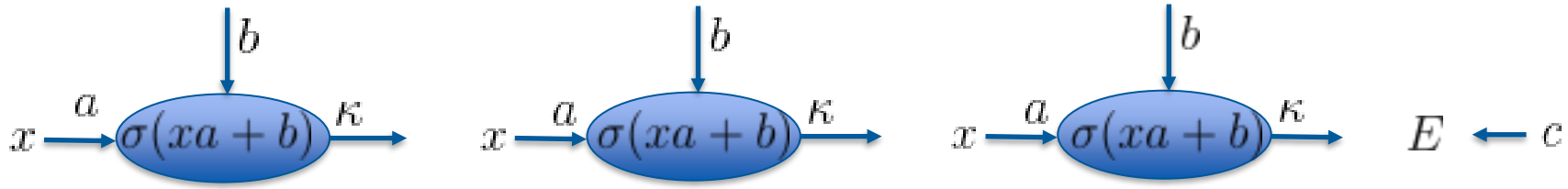
- **The AN produces the output**

$$\kappa = \sigma(\sum_i x_i a_i + b) = \sigma(\mathbf{x}\mathbf{a}^\top + b)$$

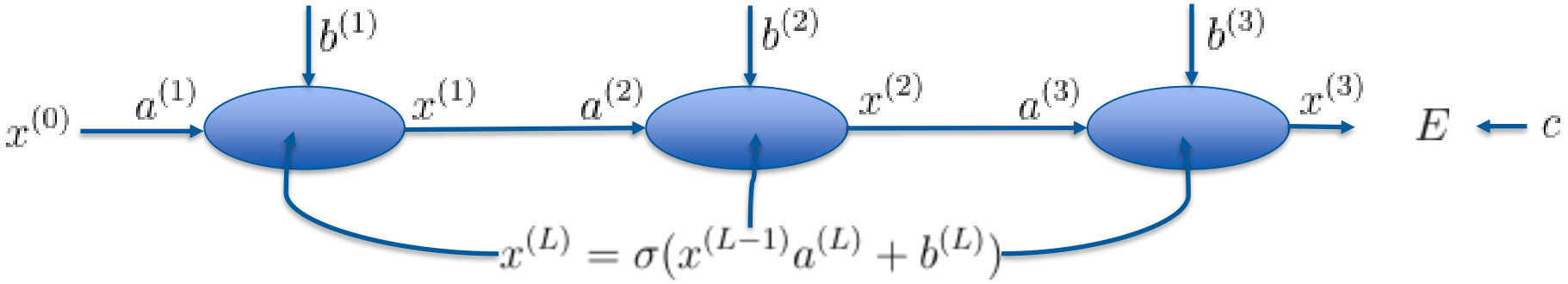
- **In fixed networks, the weights and biases a_i, b form the *model.***
- **σ is the activation function, with several classical options.**



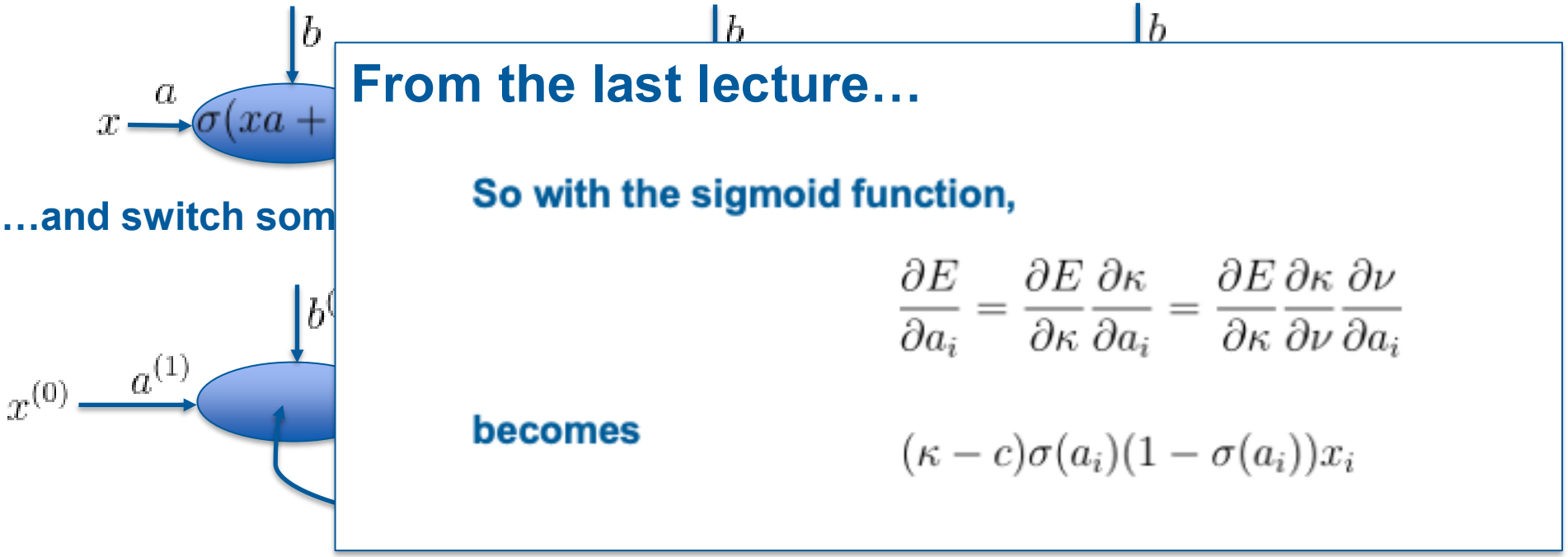
- **Let‘s start with a super simple network...**



…and switch some terms around.



- **Let‘s start with a super simple network...**



From the last lecture…

So with the sigmoid function,

$$\frac{\partial E}{\partial a_i} = \frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial a_i} = \frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial \nu}\frac{\partial \nu}{\partial a_i}$$

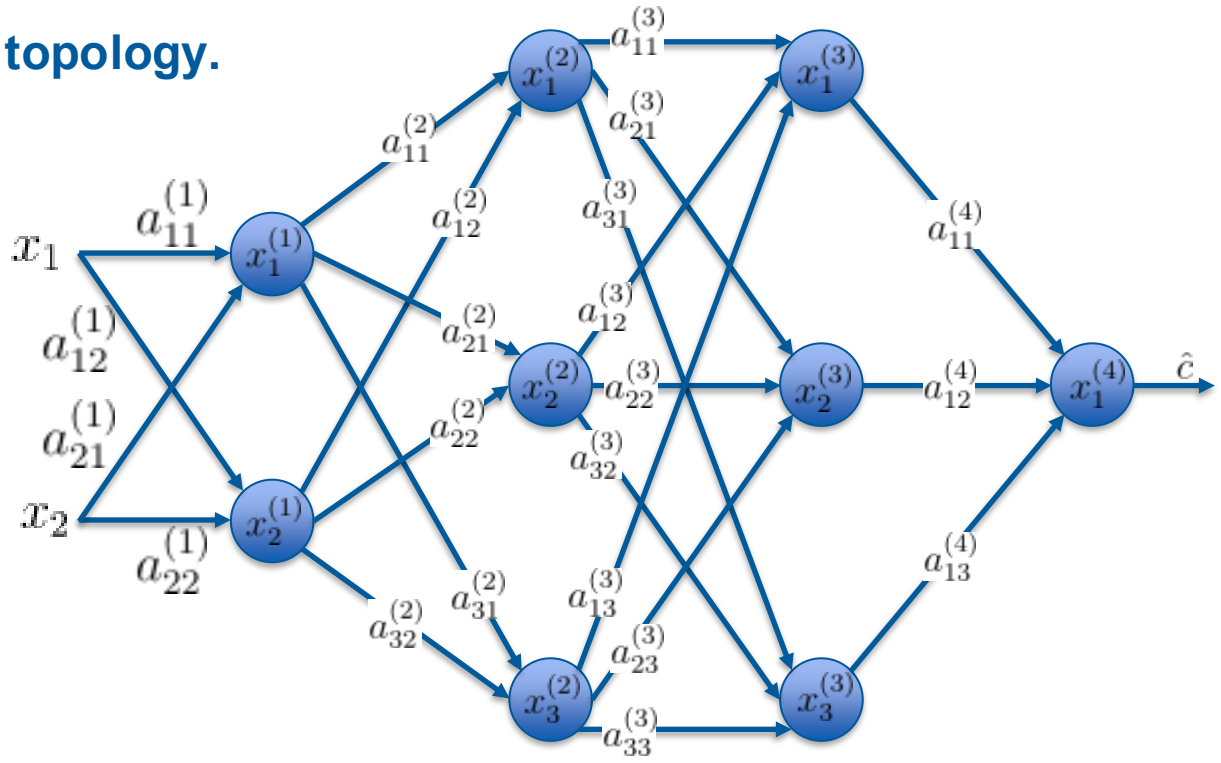
becomes

$$(\kappa - c)\sigma(a_i)(1 - \sigma(a_i))x_i$$

$$\frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial \nu}\frac{\partial \nu}{\partial a_i} = \frac{\partial E}{\partial x^{(L)}}\frac{\partial x^{(L)}}{\partial \nu^{(L)}}\frac{\partial \nu^{(L)}}{\partial a^{(L)}} = (x^{(L)} - c)\nu'(a^L + b)x^{L-1}$$

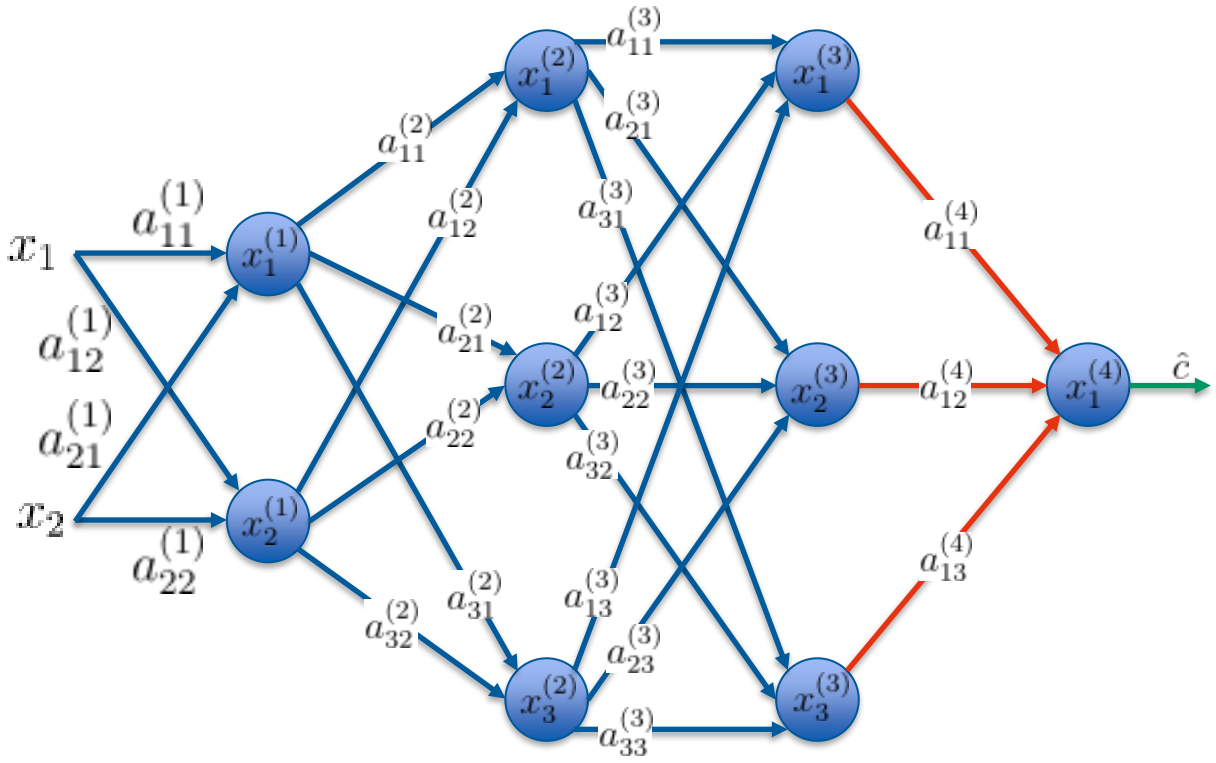
…is the error (or cost) for *one specific training example.*

- **Let‘s look at a network with a bit more complex topology.**



- **Let‘s look at a network with a bit more complex topology.**
- **The derivation for the nodes in the output layer remains the same –**

  the weights $a_{jk}^{(L)}$ are only influenced by $\hat{c} = x_i^{(L)}$.



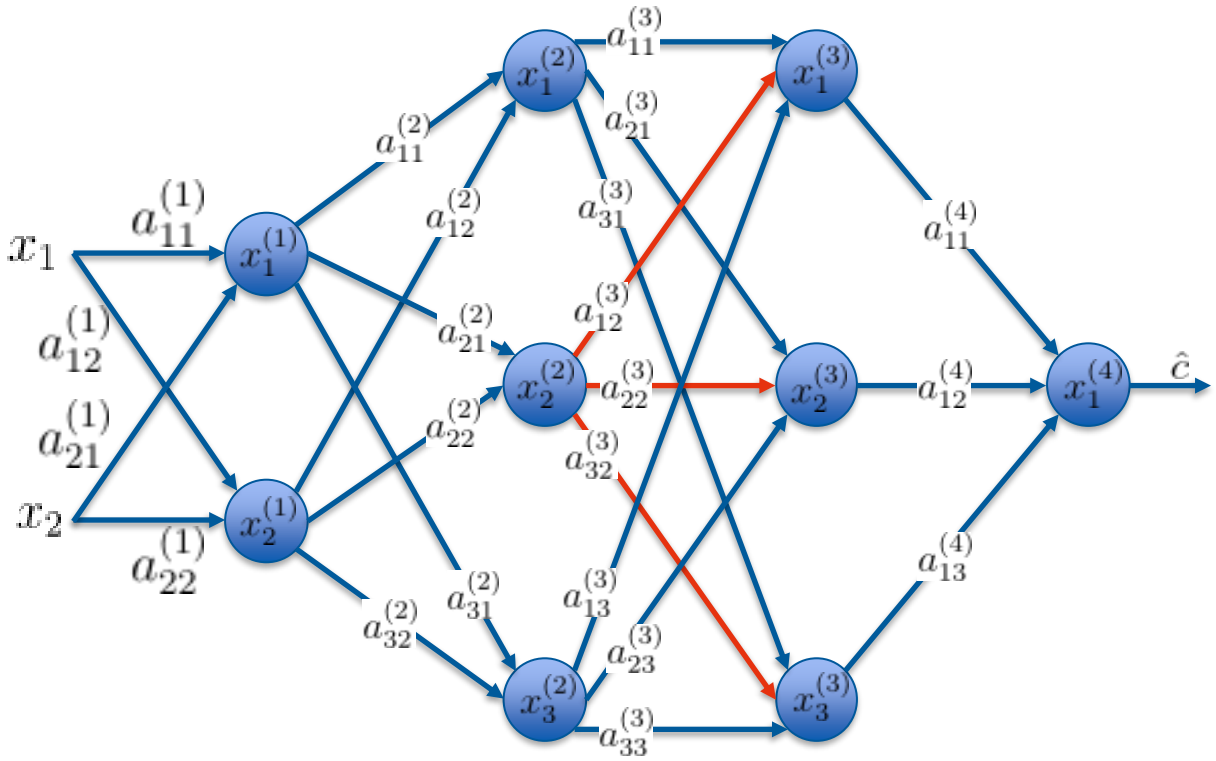
- **Let‘s look at a network with a bit more complex topology.**
- **The derivation for the nodes in the output layer remains the same –**

  the weights $a_{jk}^{(L)}$ are only influenced by $\hat{c} = x_i^{(L)}$.
- **To backpropagate, we need to consider that the local error is caused by influences coming down multiple paths:**

$$\frac{\partial E}{\partial a_{jk}^{(L)}} = \frac{\partial E}{\partial x_j^{(L)}} \frac{\partial x_j^{(L)}}{\partial \nu_j^{(L)}} \frac{\partial \nu_j^{(L)}}{\partial a_{jk}^{(L)}}$$

becomes

$$\frac{\partial E}{\partial x_k^{(L-1)}} = \sum_{j=0}^{n_{L-1}} \frac{\partial E}{\partial x_j^{(L)}} \frac{\partial x_j^{(L)}}{\partial \nu_j^{(L)}} \frac{\partial \nu_j^{(L)}}{\partial x_k^{(L-1)}}$$



- **For each example m in the training set, we can calculate an error E_m by forward-calculating the network output and subtracting it from the output pattern the network is supposed to approximate.**
- **We are interested in the performance of the network over the full training set, which we can average out:**

$$E_{\text{avg}} = \frac{1}{n}\sum_{m=1}^{n} E_m$$

$$\frac{\partial E_{\text{avg}}}{\partial x^{(L)}} = \frac{1}{n}\sum_{m=0}^{n} \frac{\partial E_m}{\partial x^{(L)}}$$

- **I.e. our gradient vector is computed as the average of the gradient vectors of the individual training examples.**

To conclude,

1. We initialize the network with random weights and biases, obtaining the vector
$$N = (a_{11}^{(1)}, a_{21}^{(1)}, \ldots, b_1, a_{12}^{(1)}, a_{22}^{(1)}, \ldots, b_2, \ldots, a_{jk}^{(N)}, b_k)^\top$$
This vector, together with the activation functions $\sigma_k^{(N)}$, describes our network.
2. We forward-compute the output error for each output neuron throughout the network by subtracting it from the setpoint output given by the training set.
3. We obtain the average error E_{avg} for all examples in the training set.
4. We compute the vector of partial derivatives as a function of the weights, biases, and average output error,
$$\nabla E = (\frac{\partial E}{\partial a_{11}^{(1)}}, \frac{\partial E}{\partial a_{21}^{(1)}}, \ldots, \frac{\partial E}{\partial b_1}, \frac{\partial E}{\partial a_{12}^{(1)}}, \frac{\partial E}{\partial a_{22}^{(1)}}, \ldots, \frac{\partial E}{\partial b_2}, \ldots, \frac{\partial E}{\partial a_{jk}^{(N)}}, \frac{\partial E}{\partial b_k})^\top$$
5. We descend the gradient given by ∇E „an ϵ-sized step“ by modifying the network:
$$N' = N - \epsilon \nabla E$$

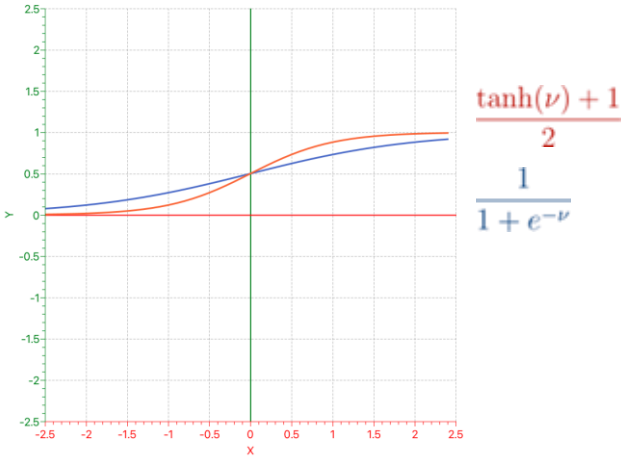
Other Activation Functions

- **Tangens hyperbolicus:**
  - tanh is a computationally effective approximation of the sigmoid function.
  - Derivative:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\frac{\partial}{\partial dx}\frac{e^x - e^{-x}}{e^x + e^{-x}} = 1 - \left(\frac{e^x - e^{-x}}{e^x + e^{-x}}\right)^2$$

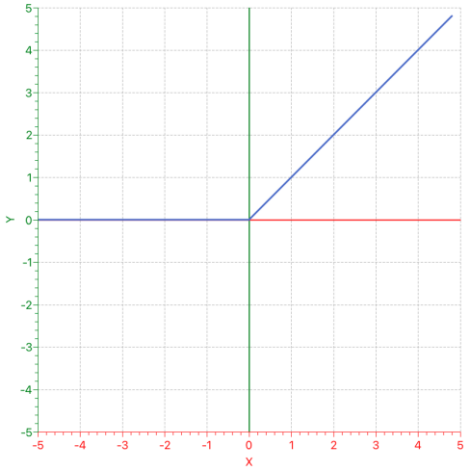
$$= 1 - \tanh(x)^2$$



Other Activation Functions

- **ReLU** $\mathrm{ReLU}(x) = max(0, x) = \frac{x + |x|}{2}; \frac{\partial \mathrm{ReLU}}{\partial x} = \begin{cases} 1 & : x \geq 0 \\ 0 & : \text{otherwise} \end{cases}$

  - Already used in 1941 as abstraction for physical neuron activation.
  - Not strictly differentiable… but convention that derivative at 0 is 1.
  - Extremely cheap to compute, allows the use of unsigned integers.
  - Nodes that do not contribute much to the network output give an output value of 0. This allows easy pruning or sparse representation of the network.
  - Strong activation is propagated (whereas sigmoid or tanh cut all activation to [0..1]).

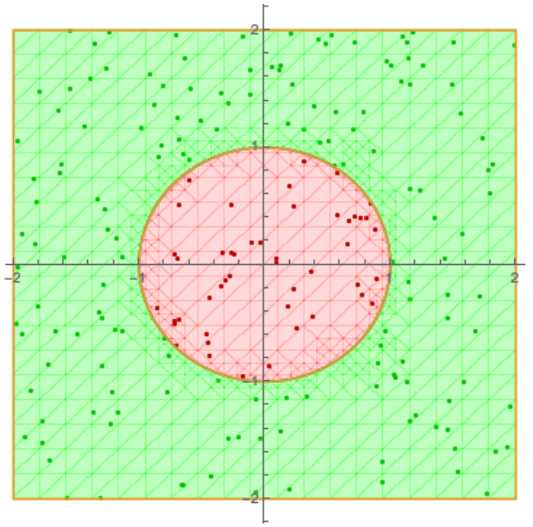


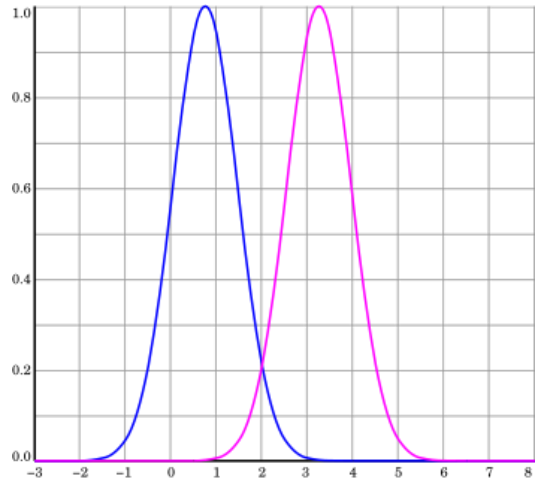
Other Activation Functions

- **Radial Basis Function (RBF):**
  - As we have seen, single perceptrons are *linear* interpolators. Depending on distribution of the examples to be classified, that may not be optimal.
  - Radial Basis Functions depict the input to the activation function φ as the distance from a center: $\varphi(\mathbf{x}) = \varphi(\|\mathbf{x} - \mathbf{c_i}\|)$
  - Usually, RBF networks are built as single-layer feed-forward networks, and their output can be written as

$$\hat{y} = f(\mathbf{x}) = \sum_{i=1}^{H} w_i \varphi_i(\|\mathbf{x} - \mathbf{c}_i\|)$$

where H is the number of radial basis functions, $\mathbf{x}$ is the network input, and $w_i, \mathbf{c}_i$ are the network parameters.



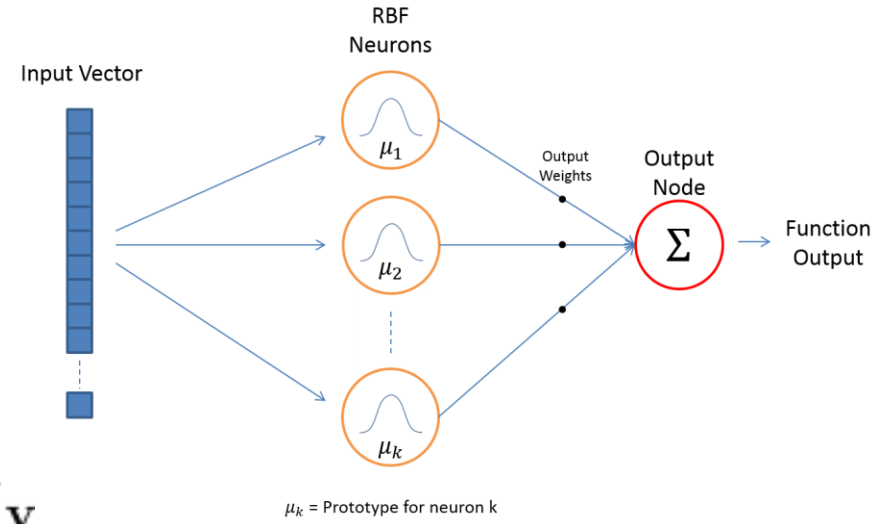


- Usually $\mathbf{c}_i$ is predetermined and only the weights w_i are learned.
- Choosing $\mathbf{c}_i$ can be realized for instance by randomly choosing H data points from the data set.
- Another method to determine the centers $\mathbf{c}_i$ is based on clustering methods, i. e. unsupervised learning.
- When $\mathbf{c}_i$ is fixed, the computation of the weights wi can be done with a least-squares method:

$$\hat{y} = f(\mathbf{x}) = \sum_{i=1}^{H} w_i \varphi_i(\|\mathbf{x} - \mathbf{c}_i\|) \Rightarrow$$

$$\mathbf{w} = (\sum_{m=1}^{M} \varphi(\mathbf{x}_m)\varphi(\mathbf{x}_m)^\top)^{-1}(\sum_{m=1}^{M} \varphi(\mathbf{x}_m)y_m) = (\Phi^\top\Phi)^{-1}\Phi^\top\mathbf{y}$$

with $\varphi(\mathbf{x}) = \begin{bmatrix} \varphi_1(\|\mathbf{x} - \mathbf{c}_1\|) \\ \varphi_2(\|\mathbf{x} - \mathbf{c}_2\|) \\ \vdots \\ \varphi_H(\|\mathbf{x} - \mathbf{c}_H\|) \end{bmatrix}, \Phi = \begin{bmatrix} \varphi(\mathbf{x_1})^\top \\ \varphi(\mathbf{x_2})^\top \\ \vdots \\ \varphi(\mathbf{x_M})^\top \end{bmatrix}, \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_M \end{bmatrix}$



- **Some examples for radial basis functions can be seen from the below table.**
- **These use α as an additional parameter to control the size and slope of the radial cluster.**
- **You can think of these like activation functions in traditional Neural Nets.**

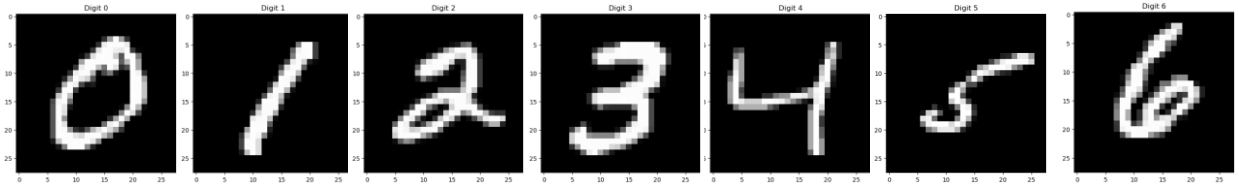
Radial Basis Function	Title
$\varphi_i(\boldsymbol{x}) = \mathrm{e}^{-(\alpha\lvert\boldsymbol{x}-\boldsymbol{c}_i\rvert)^2}$	Gaussian radial basis function
$\varphi_i(\boldsymbol{x}) = \lvert\boldsymbol{x}-\boldsymbol{c}_i\rvert^n$, where n is odd	Piecewise polynomial radial basis function
$\varphi_i(\boldsymbol{x}) = \frac{1}{1+(\alpha\lvert\boldsymbol{x}-\boldsymbol{c}_i\rvert)^2}$	Inverse quadratic radial basis function
$\varphi_i(\boldsymbol{x}) = \sqrt{1+(\alpha\lvert\boldsymbol{x}-\boldsymbol{c}_i\rvert)^2}$	Multiquadratic radial basis function
$\varphi_i(\boldsymbol{x}) = \frac{1}{\sqrt{1+(\alpha\lvert\boldsymbol{x}-\boldsymbol{c}_i\rvert)^2}}$	Inverse multiquadratic radial basis function

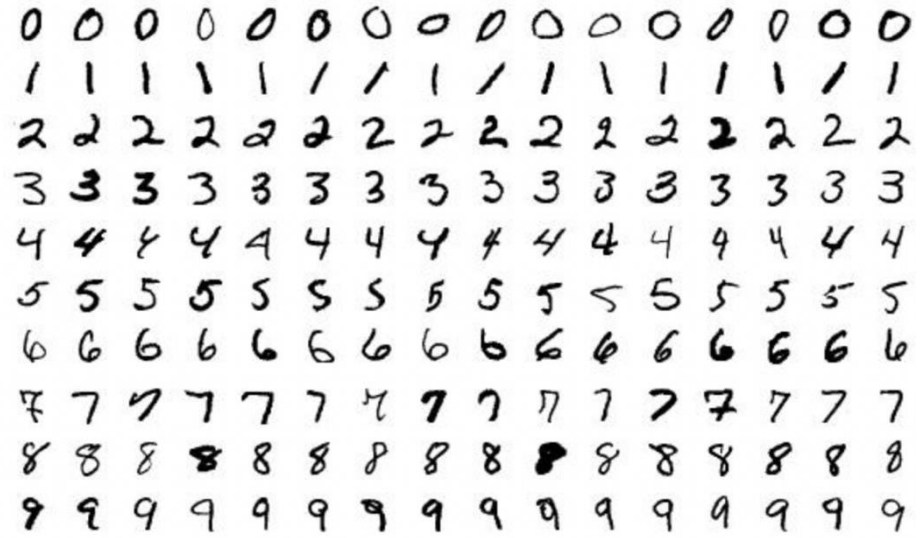
- **An important question that stays with every regression and classification tasks is the question about the number of layers and the number of neurons per layer.**
- **A feed-forward neural network with one hidden layer is already an universal approximator, i. e., every continuous function can be represented.**
- **Still, deep feed-forward artificial neural networks (with 2 or much more hidden layers) are increasingly used for practical tasks.**
- **Managing knowledge representation in deep networks is a difficult task and at the root of Verifiable / Explainable AI.**

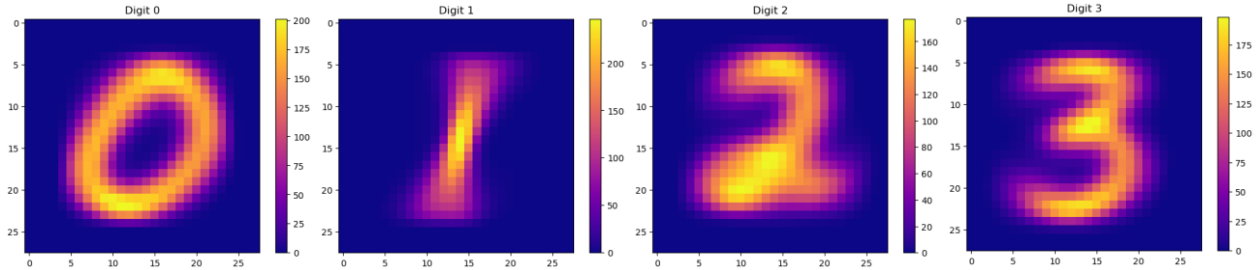
Machine Learning: A Classical Example

MNIST: Classical Example

- **The Modified NIST dataset is a standardized benchmark for ML and CV algorithms.**
- **It consists of 70,000 grayscale images of handwritten digits (0-9).**
- **The images are all normed for 28x28 pixels in size and 8-bit grayscale resolution.**
- **Each image comes with a hand-labeled ground-truth class.**
- **MNIST is an ideal beginner‘s dataset and was crucial in the development of early Convolutional Neural Networks (CNNs).**
- **We will look at two different ways to tackle it – Multilayer Perceptron and CNN.**





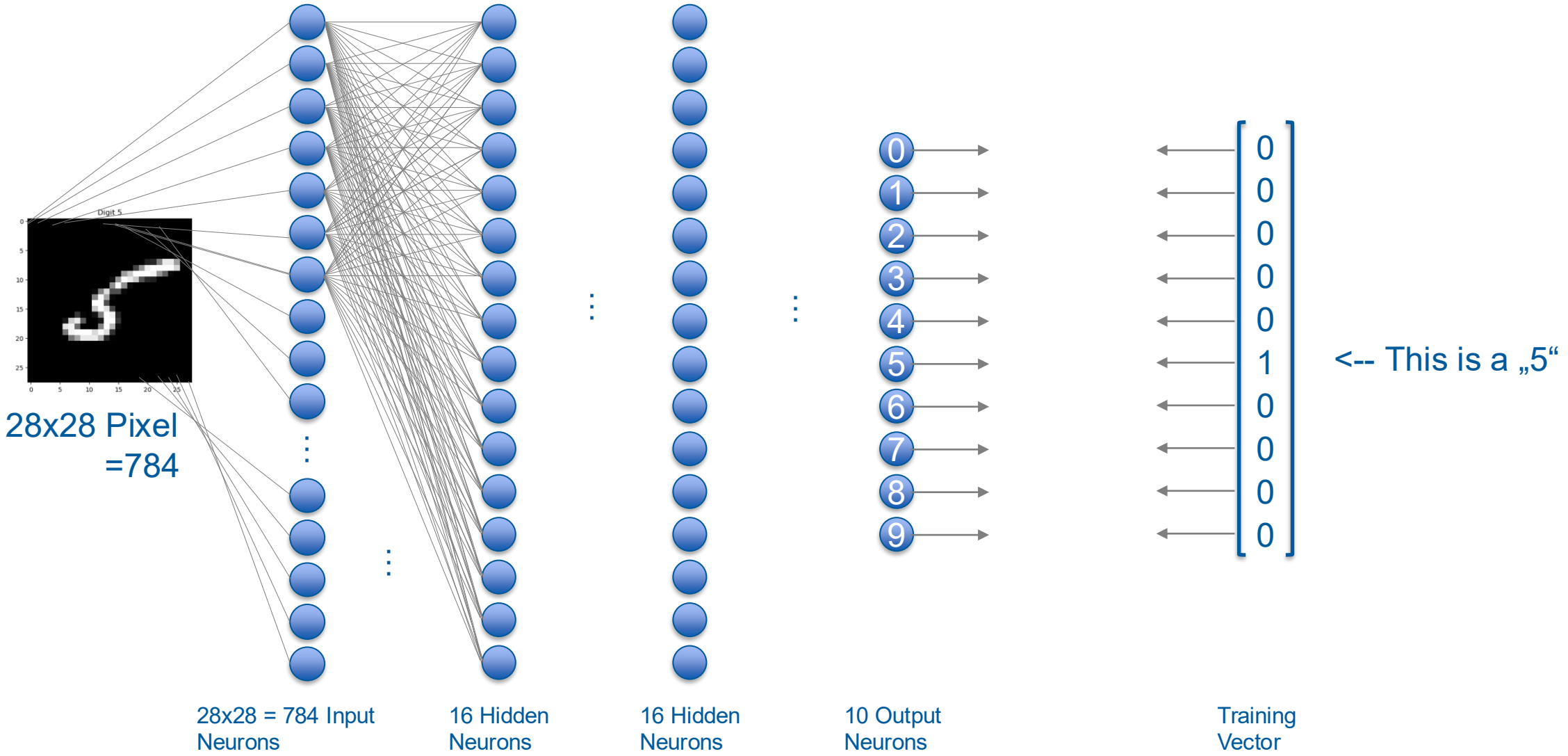


Individual images in the MNIST dataset

Heat Maps obtained by averaging all samples belonging to a certain label.

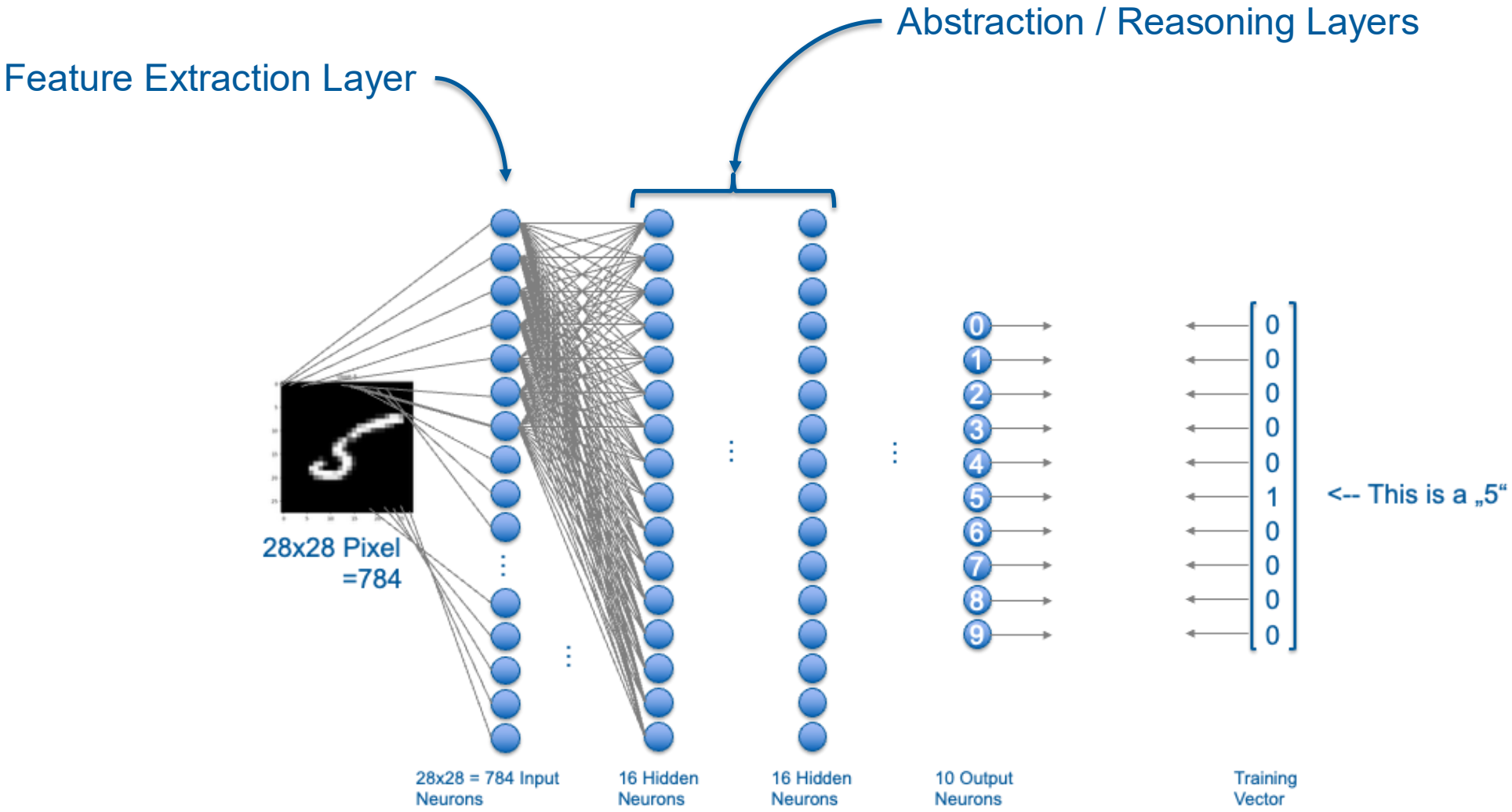
Sensor Networks and Sensor Fusion

MNIST: Classical Example



Sensor Networks and Sensor Fusion

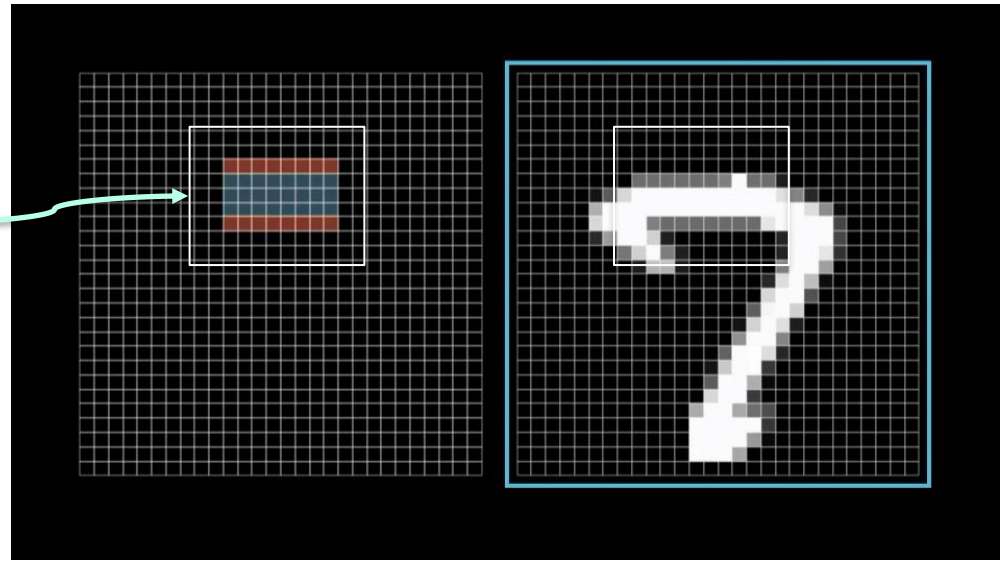
MNIST: Classical Example



MNIST: Classical Example

- **For the Feature Extraction Layer, we would expect to see a feature distribution that highlights geometric constructs describing the shapes we are training on.**

Activates if pixels under blue region are bright and surrounded by dark pixels under red region.



- **Compare this to the activation we have discussed in the Viola Jones feature set.**

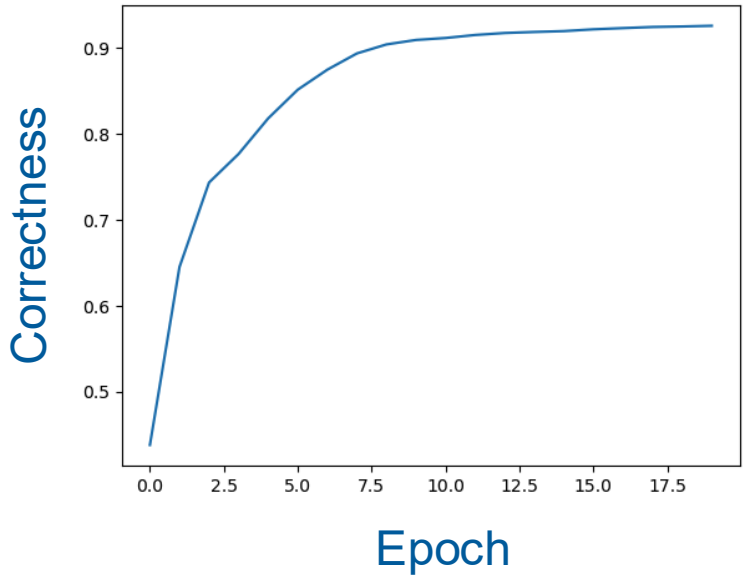
- **We can solve MNIST to „good enough“ results with a Multilayer Perceptron.**
- **In our example, we will use a variant with two hidden layers of 16 neurons each.**
- **This gives us a network of 13,002 weights and biases.**
- **The network has 10 outputs that are all in $[-1..1]$ if Sigmoids are used, or $[0..\infty]$ if ReLU is used.**
- **We can select the best result using $i_{\max} = \operatorname{argmax}_i y_i$.**
- **Alternatively we can use the softmax function:**

$$\operatorname{softmax}(\mathbf{y}) = \frac{e^{y_i}}{\sum_i e^{y_i}}$$

- It normalizes the resulting set into a probability distribution ($\sum y_i = 1$),
- And it amplifies the maxima in the input tuple exponentially.

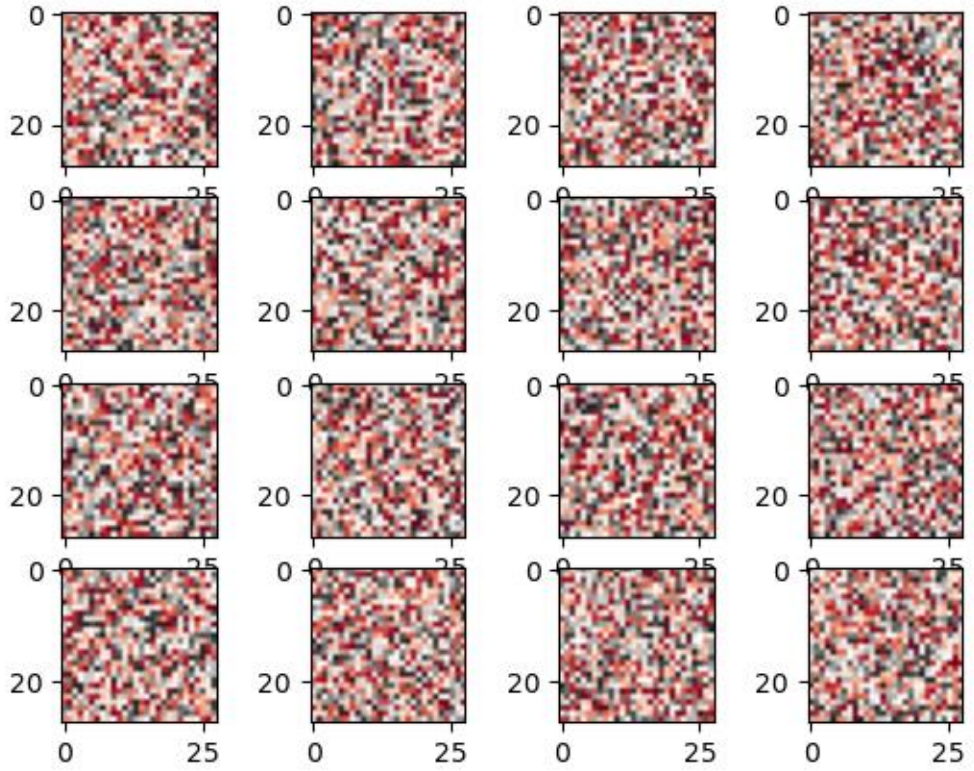
MNIST: Classical Example

- **We will walk through the training session using Jupyter Notebook.**
- **You can find the notebook on Moodle, the filename is `MNIST_MLPerceptron.ipynb`.**
- **Training should converge to >90% after about 10 epochs. But try it several times and see if you get different output curves.**
- **Also play with substituting ReLU for the Sigmoid activation, and see how that changes things! Again, you may need to do this several times to get a statistical sample.**

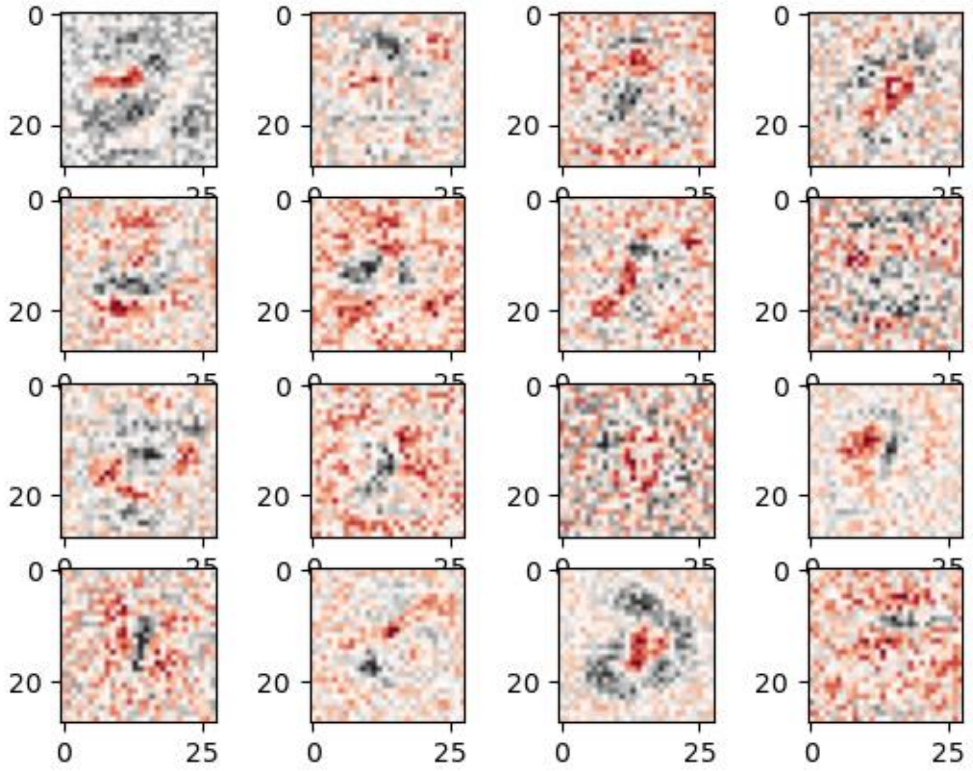


Machine Learning: MNIST Example

- **Visualization of layer 0 weights („Feature Extraction“) at beginning and end of the training.**



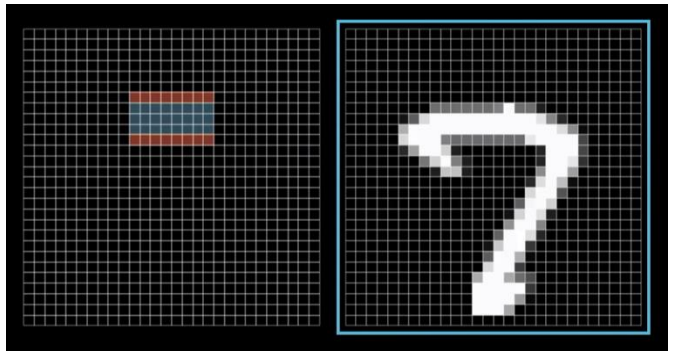
Beginning of training – weights randomized

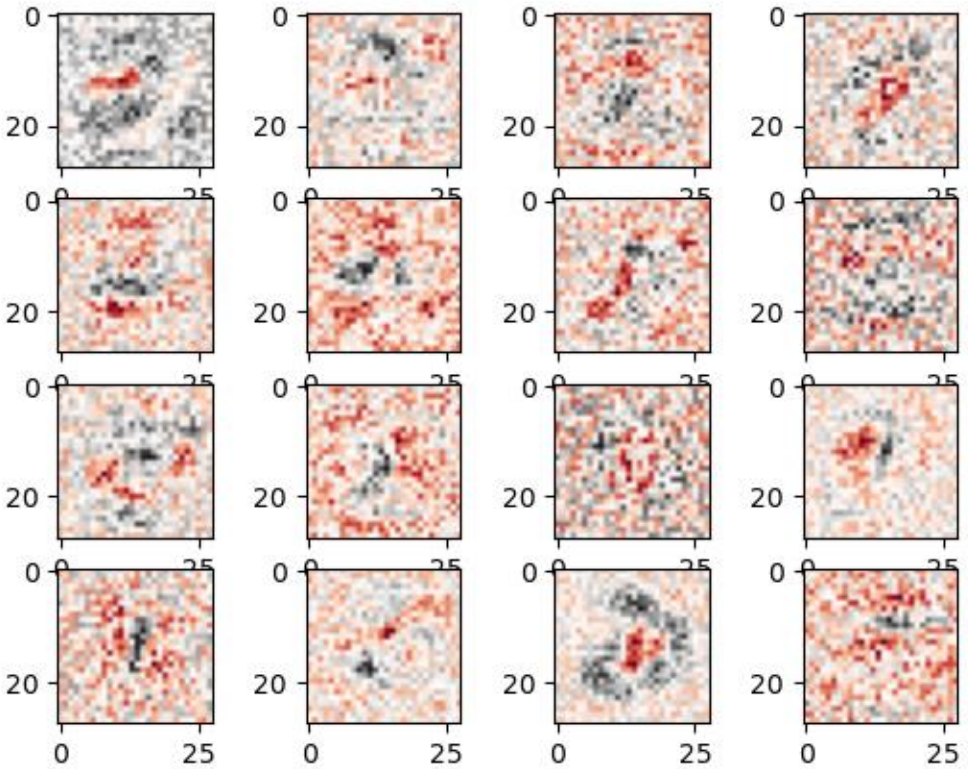


End of training – weights converged

MNIST: Classical Example

- **Contrast our expected feature distribution with the actual one.**
- **We see feature clusters. Can you relate them to the numbers we have been training on?**
- **Does the output match your expectation?**
- **If the network did not perform as expected, how would you**
  - Diagnose
  - Explain
  - Correct?





- **The multilayer perceptron performs well for the MNIST database, as we have seen.**
- **However it has several weaknesses for image recognition.**
- **It is sensitive to scale, rotation and translation of the input images, and has too many parameters to be useful for scanning entire images.**
- **This makes it impractical for tasks in the 3D world, for example.**
- **We can put some more intelligence into the system.**
- **The combination of image convolution and Deep Learning leads to Convolutional Neural Networks.**



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!